Quiero que me ayudes a generar la lógica fake del teléfono de mi proyecto PBIO.

En este prompt quiero trabajar solamente con la parte de Android que se comunica con el servidor REST.

No quiero hacer aquí Bluetooth, ni escaneo, ni decodificación del iBeacon, ni la interfaz.

Tampoco quiero hacer la lógica real del backend.

La idea es:

Android
  ->
lógica fake
  ->
REST
  ->
lógica real
  ->
BBDD

La llamamos lógica fake porque desde Android queremos poder utilizar una operación como:

medicion: Medicion
-->
registrarMedicion()
-->
Medicion

pero realmente esta función no guarda nada directamente.

Por dentro tiene que hacer:

POST /api/mediciones

MEDICIÓN

La medición que recibe esta capa tendrá:

uuid
nombre_dispositivo
tipo

valor
contador
rssi

Conceptualmente:

```
Medicion = (
    uuid: Text,
    nombre_dispositivo: Text,
    tipo: Text,
    valor: Z,
    contador: N,
    rssi: Z
)
```

RSSI puede faltar.

Android no tiene que enviar:

id_medicion
id_dispositivo
fecha_hora

REGISTRAR MEDICIÓN

Quiero:

```
medicion: Medicion
-->
registrarMedicion()
-->
Medicion
```

Esta función tiene que:

1. recibir la medición;
2. convertirla a JSON;
3. hacer POST /api/mediciones;
4. esperar la respuesta sin bloquear la interfaz;
5. interpretar el JSON que devuelve el servidor;
6. devolver el resultado al resto de la aplicación.

No quiero que esta función haga INSERT ni acceda directamente a la base de datos.

Eso lo hace la lógica real del backend.

JSON

El JSON que se manda será parecido a:

```
{
   "uuid": "EPSG-GTI-PROY-3A",
   "nombre_dispositivo": "GTI3A-2025",
   "tipo": "CO2",
   "valor": 123,
   "contador": 27,
   "rssi": -68
}
```

No quiero mandar:

id_medicion
id_dispositivo
fecha_hora

CÓDIGO EXISTENTE

Mira primero el proyecto Android que ya tenemos.

Hay código antiguo del profesor para hacer peticiones REST y puede existir una clase PeticionarioREST.

Si sirve, reutilízala.

Si tiene errores, corrígelos.

Revisa sobre todo:

- que no bloquee el hilo principal;
- códigos HTTP de error;
- errores de conexión;
- timeout;
- JSON incorrecto;
- codificación UTF-8;
- cierre correcto de recursos.

RESPUESTAS

Quiero gestionar:

201 -> medición registrada

400 -> datos incorrectos

404 -> recurso no encontrado si alguna operación lo necesita

405 -> método incorrecto

500 -> error del servidor

También:

servidor apagado

timeout

fallo de conexión

respuesta vacía

JSON incorrecto

Si pasa algo de esto, la aplicación no se tiene que cerrar.

Quiero que la lógica fake devuelva el error a quien la haya llamado.

No quiero que la clase REST sea la que decida directamente qué texto enseñar en pantalla.

SEPARACIÓN

No quiero que MainActivity tenga que hacer manualmente:

- construir URLs;
- montar JSON;
- hacer peticiones HTTP;
- interpretar respuestas.

Quiero que pueda hacer conceptualmente algo como:

logicaFake.registrarMedicion(medicion)

y que esta capa se encargue de todo lo relacionado con REST.

URL

Quiero la URL base del servidor en un único sitio.

No repitas la misma dirección en varias funciones.

Después tiene que ser fácil cambiar:

http://servidor/...

La ruta que necesitamos ahora es:

/api/mediciones

COMENTARIOS

Quiero comentarios en español.

No pongas comentarios obvios en cada línea.

Quiero cabeceras con nuestra notación en las funciones importantes.

Por ejemplo:

```
// -----------------------------------------------
// medicion: Medicion --> registrarMedicion() --> Medicion
//
// Solicita al servidor que registre una medición.
// -----------------------------------------------
```

Otro:

```
// medicion: Medicion --> convertirMedicionAJSON() --> Text
```

Otro:

// respuesta: Text --> interpretarRespuestaMedicion() --> Medicion

Usamos:

N = naturales
Z = enteros
R = reales
B = booleanos
Text = texto

Si en Java necesitas callbacks porque la petición es asíncrona puedes usarlos, pero no hace falta poner esos detalles en la firma lógica de la cabecera.

TESTS

Quiero tests automáticos para esta parte.

No quiero depender de que el servidor real esté encendido para todos los tests.

Utiliza mocks o respuestas simuladas.

Quiero probar como mínimo:

1. Crear correctamente el JSON de una medición.

2. Que el JSON tenga uuid.

3. Que tenga nombre_dispositivo.

4. Que tenga tipo.

5. Que tenga valor.

6. Que tenga contador.

7. Que tenga rssi cuando existe.

8. Que no mande id_medicion.

9. Que no mande id_dispositivo.

10. Que no mande fecha_hora.

11. Probar una medición sin RSSI.

12. Comprobar que se utiliza POST.

13. Comprobar que se utiliza /api/mediciones.

14. Simular respuesta 201.

15. Simular 400.

16. Simular 500.

17. Simular servidor apagado.

18. Simular timeout.

19. Simular JSON incorrecto.

20. Comprobar que un error no cierre la aplicación.

ANTES DE PROGRAMAR

Antes de cambiar nada:

1. Mira el proyecto Android.
2. Busca el código REST que nos dio el profesor.
3. Dime qué podemos reutilizar.
4. Dime qué errores tiene.
5. Dime qué archivos vas a crear o modificar.
6. Comprueba que no estás metiendo BLE ni interfaz.
7. Comprueba que no estás haciendo la lógica real del backend.

Después implementa la lógica fake y los tests.

Al terminar dime:

- qué archivos has creado;
- qué archivos has modificado;
- qué código del profesor has reutilizado;
- cómo funciona registrarMedicion();
- qué JSON se manda;
- cómo gestionas los errores;
- qué tests has creado;
- cuáles pasan;

- cuáles fallan;
- y qué problemas has encontrado.

No hagas Bluetooth, interfaz Android, SQL, backend, servidor REST ni navegador.

Quiero solamente la lógica fake del teléfono.